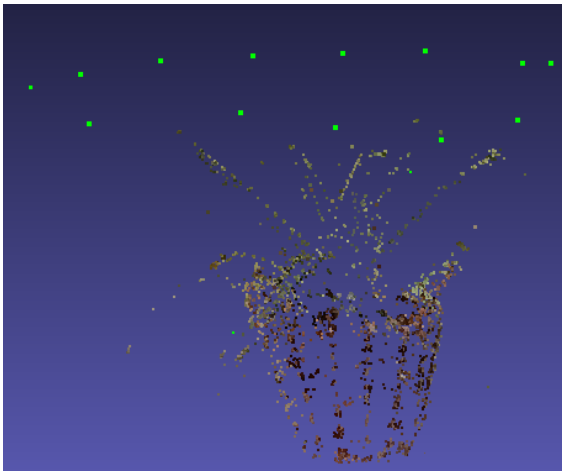
*3D DATA PROCESSING - LAB 2 (**GROUP ASSIGNMENT**)*



Topic: Structure from Motion
Goal: Estimate the 3D structure of a small scene from a sequence of images with some field-of-view overlaps. Evaluate your reconstruction system on the provided datasets as well as on one or more datasets captured with your smartphone.
Groups: Students should organize in groups of two members and do the assignment together. Only one submission per group is sufficient (just clearly indicate the names and the "matricola" (IDs) of the components in the report).

Introduction

For this assignment, it is required to complete and/or modify some code portions of a simple Structure from Motion pipeline. This pipeline includes two main modules:

- A feature extractor and matcher module represented by the **`matcher`** app defined in **`matcher_app.cpp`** (do not modify this source file). This application requires as input the calibration parameters of the camera used to acquire the images, a folder containing the images used for the 3D reconstruction, the name of the file where the processed data will be stored, and a flag specifying whether classical or modern visual features will be used. This file will be filled with all the information needed to build the **scene graph** (e.g., the collected observations) that will be used in the subsequent reconstruction module.

- An incremental structure from motion module represented by the **`basic_sfm`** app defined in **`sfm_app.cpp`** (do not modify this source file), which requires as input the output file produced by the matcher app and the name of the **`.ply`** (Polygon File Format) file in which to save the point cloud resulting from the 3D reconstruction.

Groups are required to complete/modify some portions of code within the **`features_matcher.cpp`** source file, namely:

1) Perform Geometric Verification (for both Essential and Homography models) of extracted feature matches, possibly discarding the outliers (task 1/7, function **`exhaustiveMatching()`**).
2) Extract salient points + descriptors from each image. A standard implementation that uses the ORB features is already provided. You are required to implement an alternative, more modern feature detection and description scheme. Examples are:
    a) SuperPoint: https://github.com/eric-yyjau/pytorch-superpoint
    b) DISK: https://github.com/cvlab-epfl/disk
    c) ALIKED: https://github.com/Shiaoming/ALIKED

    or other alternatives (task 2/7, function **`extractFeatures()`**).

    IMPORTANT: You also need to update the matching part in the **`exhaustiveMatching()`** function.

and complete/modify some portions of code within the **`basic_sfm.cpp`** source file, namely:

1) Extract the essential matrix E and the homography matrix H, and check the number of inliers for both models to extract the seed pair. In a good seed pair, the number of inliers for E should be greater than the number of inliers for H. For a suitable pair, recover from E the initial rigid-body transformation and check whether the recovered transformation is primarily a sideward motion. In case, promote the pair as a seed pair, setting the estimated transformation as the initial rigid body transformation (task 3/7, function **`incrementalReconstruction()`**);
2) Triangulate a set of newly registered 3D points from a pair of observations (2D projections) (task 4/7, function **`incrementalReconstruction()`**);
3) Implement an auto-differentiable cost function for the Ceres Solver problem (task 5/7, **struct `ReprojectionError()`**);.
4) Add a residual block inside a Ceres Solver problem (task 6/7, function **`bundleAdjustmentIter()`**). To handle outlier matches, you are required to experiment with different robust loss kernels. Compare the standard '**`NULL`**' loss (least squares) with robust alternatives.
5) Check at each registration iteration if the reconstruction has diverged. In this case, just reset the reconstruction and start from scratch with a new pair of seeds (task 7/7, function **`incrementalReconstruction()`**).

Supplementary work for groups of three students (in `basic_sfm.cpp`): Implement alternative seed selection and next-best view selection strategies (task OPTIONAL 1 in function **`solve()`** and task OPTIONAL 2 in function **`incrementalReconstruction()`**).

Note: In **`basic_sfm.cpp`**, all observations (2D projections) are **normalized**, i.e., they refer to the canonical camera, i.e., a camera with matrix K equal to the identity and no distortion.

Detailed descriptions of what is required are reported directly inside the code. Chunks of code to be completed are marked with the preamble:

```
//////////////////// Code to be completed (x/y) /////////////////
```

where `x/y` denotes the task (e.g., 1/7, …, 7/7) followed by a description of the required functionalities.

Please check the README and code carefully, read all the comments, and try to understand the flow.

What you need to deliver

- Source code (without objects and executables)
- One or more datasets captured with your camera (avoid using excessively high-resolution images; rescale them if necessary), and the corresponding camera calibration parameters. To calibrate your camera and produce the calibration parameter file for the **`matcher`** app, follow the instructions shared on the course webpage.
- The obtained point clouds (**`.ply`** files) for all datasets.
- A short written report with:
    - A brief description of the work done, and the problems encountered, if any, clearly specifying who did what;
    - Some qualitative results on the two provided datasets and on the datasets acquired with your smartphone. The results in the report can be shown through point-cloud screenshots (for example, you can use MeshLab to visualize the produced .ply file, see, for instance, the figure above): they should be "reasonably good" and make sense with the framed scene.
    - Comprehensive tables comparing reconstruction performance across all datasets (the provided sequences and your own smartphone acquisitions). You are required to evaluate the **standard ORB baseline** against your **modern feature implementation**, as well as the impact of robust loss kernels (e.g., Cauchy or Huber) against standard least squares.
      The tables must report the following metrics:
        - **Reconstruction Density**: The total number of valid 3D vertices (points) generated in the final .ply reconstruction.
        - **Success Rate**: The registration efficiency, defined as the ratio of successfully localized camera poses to the total number of images in the input sequence.
        - **Average Reprojection Error**: The mean geometric error (measured in pixels) between the observed 2D feature points and the 2D projections of the corresponding 3D points.

    - **Supplementary work for groups of three students** (please contact me in advance if this applies): in addition to the standard activities, these groups are also expected to quantitatively evaluate in the tables above the effect of their **custom-built seed pair selection and the next-best view selection** strategies.

HINTS

1) OpenCV provides several utility functions for geometry and 3D reconstruction:

https://docs.opencv.org/4.2.0/d9/d0c/group__calib3d.html

Among others:

`void cv::Rodrigues ()`

convert to axis-angle rotation representation and vice-versa.

`Mat cv::findEssentialMat()`
`Mat cv:: findHomography()`

Estimate the Essential matrix and the Homography matrix from two sets of correspondences

`int cv::recoverPose()`

Recovers the relative camera rotation and the translation from an estimated essential matrix

…

2) Read the Ceres Solver bundle adjustment tutorial carefully:

http://ceres-solver.org/nnls_tutorial.html#bundle-adjustment